4.1 High Priority (Affects Tournament Results)
FU-1: Target Core Count for Phase 1 Deployment
Priority: High
Context: config.cfg line 16 vs. Q-13 contract answer
Question: The Q-13 answer specifies '24 physical CPU cores (AMD Genoa or equivalent)' as the target deployment, but config.cfg currently uses TOTAL_CORES=64. Which value is authoritative for Phase 1 benchmarking? This affects DQN normalization (C-11), SA core bounds (Q-6), and malleable scheduling math.
Answer:

Please see this edge node config file in github folder
https://github.com/KanwalBat00l/KairosSNNI/blob/main/Howto.md

Regarding **FU-1: Target Core Count**, follow this authoritative instruction:
Authoritative Value: 24 Cores is the target deployment for Phase 1 to represent an edge node.
Technical Requirements:

1. **Configuration:** Update config.cfg (line 16) to TOTAL_CORES=24.
2. **DQN Normalization (C-11):** Ensure the state vector normalization logic uses **24.0** as the denominator for the CPU feature. If the logic is configuration-driven (as per C-11), it will now correctly scale to the $P = 24$ hardware pool.
3. **SA Search Space (Q-6):** The Simulated Annealing engine must restrict the malleable core-partitioning search to the range $[1, 16]$ cores per job, ensuring that the total system sum does not exceed **24**.
4. **Scientific Reasoning:** Our research specifically targets **Edge Server** scenarios (like the AMD Genoa 24-core nodes). Benchmarking on 64 cores would dilute the "Memory-Compute Pressure" results, as the 35GB VGG16 bottleneck is more scientifically significant when it occupies a larger percentage of a 24-core system's relative capacity.

Summary: Please align all internal logic, benchmarks, and configuration files to a **24-core** environment. This is the hardware baseline for the "Scheduling Tournament" and the final paper results.

FU-2: DQN Bootstrap Strategy for Tournament
Priority: High
Context: C-08, C-19, Q-13 tournament criteria
Question: For the Phase 1 Scheduling Tournament (Q-13), the DQN scheduler requires pre-trained weights to function (otherwise it falls back to LST). Should DQN be pre-trained on FCFS/EDF traces first (bootstrapped), or should the 20% Psi improvement target only apply to SA vs. FCFS, with DQN evaluated separately? How many training episodes are considered sufficient for a fair DQN evaluation?
Ans:
Regarding **FU-2: DQN Bootstrap Strategy**, please follow these instructions for the Phase 1 Scheduling Tournament:
1. Bootstrapping and Training Strategy

The DQN **must be pre-trained** (bootstrapped) using existing traces from our **Urgency-Based** algorithms (specifically **EDF**).

- **State Features:** Use the **Profiler** (TAT & resource predictor) to provide the input features ($E_{i,c}$ and M_i). This gives the DQN the necessary knowledge of job costs.
- **Method:** Use **Imitation Learning** on EDF traces. This ensures the agent starts with a logical baseline before exploring the "Opportune" (Kairos) space.
- **Volume:** Perform **500 to 1,000 training episodes** (1 episode = 1,000 requests) until the reward curve stabilizes.

2. Tournament Classification and Targets

The 20% Ψ improvement target **applies to both Intelligent Optimizers** (SA and DQN) relative to the **Baselines**.

- **Baselines:** FCFS and SJF+Aging.
- **Urgency-Based:** EDF and Least Slack Time (LST).
- **Intelligent Optimizers (The Goal):**
    - **Simulated Annealing (SA):** Must utilize **3D perturbation** (Swapping, Sizing, and Dwell-shifting) to maximize Ψ.
    - **DQN:** Must implement the **Pointwise Ranking policy** to determine the optimal job sequence.

3. Reward Function for DQN

The DQN reward must be tied directly to the **Utility Metric (Ψ)**.

- **Reward:** $+1.0$ for successful SLO attainment ($\chi = 1$).
- **Penalty:** -1.0 for every SLO violation ($\chi = 0$).
- **Efficiency:** Add a small negative penalty based on TAT to minimize the denominator of Ψ.

Summary:

Bootstrap the DQN on **Urgency-Based (EDF)** traces using **Profiler** data as the state. Train for ~1,000 episodes. Both **SA (with 3D perturbation)** and **DQN (with Pointwise Ranking)** are expected to beat the **FCFS baseline** by at least 20% in the final Ψ score.

FU-3: Malleable Core Speedup Profiles

Priority: High

Context: SAScheduler.hpp lines 181-185, contract page 3

Question: The SA 4D perturbation (Q-6) adjusts core counts for jobs, but the cost function uses a linear speedup model (speedup = job_cores / default_cores). The contract states models have 'nonlinear speedup profiles.' Can the client provide empirical speedup data per model at different core counts (e.g., 1, 2, 4, 8, 16 cores)?

Answer:

Regarding **FU-3: Malleable Core Speedup Profiles**, please update the cost function logic in the **Request Scheduler**. The previous linear speedup model is incorrect. You must implement a sub-linear model based on **Amdahl's Law**.

In SNNI, the speedup is limited by the serial part of the protocol. The offline phase (e_{pre}) is strictly single-core. It cannot be parallelized. Only the online phase (e_{on}) can use multiple cores.

1. The Revised Speedup Formula

Calculate the total execution time (T) for p cores as follows:

T(p) = e_pre + [(theta_c * e_on) / (p^k)]

- **e_pre:** This is a fixed constant from the Profiling Table. It does not change when you add cores.
- **e_on:** This is the baseline online time. It is multiplied by the client slowness factor (θ_c).
- **p:** The number of cores assigned to the job (1 to 16).
- **k:** The parallel efficiency factor. Use **k = 0.8** as the default. This represents sub-linear scaling due to communication overhead.

2. Why this is important for the Scheduler

Because e_{pre} is a constant bottleneck, adding more cores results in **diminishing returns**.

- **For Large Models (like VGG16):** e_{pre} is very high (165 seconds). Even if you make the online phase run at infinite speed with 16 cores, the total time will never drop below 165 seconds.
- **For Small Models (like M1):** e_{pre} is low. Adding cores has a much bigger impact on the total speedup.

3. Implementation Actions

- **Update SAScheduler.hpp:** Replace the linear speedup = job_cores / default_cores logic with the formula above.
- **Update Cost Function:** Use this new $T(p)$ to predict the **Virtual Finish Time (VFT)** and calculate the **Psi (Ψ)** metric for different core counts.
- **Optimization:** This formula allows the **SA and DQN** agents to realize that giving 16 cores to a "Heavy" model is often a waste of resources. It encourages the system to save those cores for "Light" models where they provide more value.

Summary:

The offline phase is a "fixed cost." Only the online phase scales with cores, and it scales with an efficiency of 0.8. Please use this model to find the most **opportune** core allocation.

4.2 Medium Priority (Affects Correctness)
FU-4: Centralized Triple Check (C-31)
Priority: Medium
Context: IScheduler.hpp popReadyJob(), KairosServer.cpp dispatcher
Question: The contract's C-31 answer states 'Centralized Triple Check: popReadyJob() must verify File Readiness, Core Gaps, and Memory Gaps before popping.' The current implementation only checks file availability in popReadyJob() and defers core/memory checks to the dispatcher. Should the implementation be refactored to match the contract's centralized specification?
Answer:

Regarding **FU-4: Centralized Triple Check**, follow these technical instructions to ensure the implementation aligns with the **KairosSNNI** architectural requirements:

1. Refactoring Decision: Mandatory Centralization

Yes, the implementation **must be refactored** to centralize the Triple Check within IScheduler::popReadyJob().

Scientific Reason: In a non-preemptive system with a 35GB memory wall, the "Brain" (Scheduler) must only release a job to the "Muscle" (Dispatcher) when it is **physically executable**. Deferring checks to the dispatcher creates a "False Readiness" state, where the scheduler might think it has solved the queue, but the dispatcher is actually stalling on resource gaps. Centralizing this ensures the **Non-Blocking HoL Mitigation** works across the entire multi-tenant workload.

2. Implementation Requirements for popReadyJob()

The base class implementation in IScheduler.hpp must be updated to perform the following atomic verification loop:

- **Check 1: Cryptographic Readiness:** Query the PreprocessedFileManager to ensure the model-specific J_{pre} material exists in the 1TB Project Space.
- **Check 2: Spatial Capacity:** Query the ResourceManager to ensure that the exact number of cores (p_J) assigned during optimization is currently free.
- **Check 3: Memory Safety:** Query the ResourceManager to ensure that current_usage + m_on does not exceed the **90% RAM Threshold** (M).

3. The Bypass Logic (HoL Mitigation)

If a job fails **any** of the three checks:

1. **Do not pop it.** Leave it in its position in the queue.
2. **Bypass:** Immediately proceed to the next request in the sorted permutation.
3. **Dispatch:** Only the first request that passes **all three checks** is popped and returned to the dispatcher.

4. Impact on Dispatcher Logic

By refactoring this, the Dispatcher's role in KairosServer.cpp becomes significantly leaner. It simply calls popReadyJob(), and if a job is returned, it is **guaranteed** to have the cores, memory, and files ready for immediate **Strict Physical Core Pinning**.

Summary for Task List:

- **Modify IScheduler::popReadyJob():** Integrate the ResourceManager and PreprocessedFileManager calls into the bypass loop.
- **Remove redundant checks:** Clean up the manual resource verification logic inside the Dispatcher loop in KairosServer.cpp.
- **Logging:** Ensure the Logger records a "Bypass Event" specifically identifying which of the three checks failed (e.g., BYPASS_REASON_MEMORY or BYPASS_REASON_FILE).

This ensures that every dispatch decision is "Opportune." It prevents the system from locking a core for a job that might immediately crash due to a missing file or memory overflow.

FU-5: Welford Dual-Phase Tracking (C-32)
Priority: Medium
Context: Types.hpp ModelProfile, C-32
Question: The contract says 'track and store the Standard Deviation (sigma) for both phases.' ModelProfile currently has only one Welford tracker. Should there be separate Welford trackers for

pre-processing variance (sigma_pre) and inference variance (sigma_on)?
Answer:
Regarding **FU-5: Welford Dual-Phase Tracking**, follow these technical instructions to ensure the **Telemetry Engine** captures the high-fidelity data required for our stochastic analysis.

1. Tracking Decision: Mandatory Dual-Phase Tracking

Yes, you must implement **separate Welford trackers** for the preprocessing phase (σ_{pre}) and the online inference phase (σ_{on}).

2. Technical Reasoning

In SNNI, the sources of variance for the two phases are fundamentally different and must not be aggregated:

- **Preprocessing Variance (σ_{pre}):** Driven by server-side factors such as disk I/O latency (reading/writing 35GB files) and OS-level context switching. It is generally stable but subject to "bursty" interference.
- **Inference Variance (σ_{on}):** Driven by external factors such as network jitter (WAN/LAN) and client-side processing variability (θ_c). This phase is much more volatile.
- **The Problem:** Mixing these into a single σ would create a mathematically "noisy" value that accurately describes neither phase, making our **Virtual Finish Time (VFT)** estimations unreliable.

3. Implementation Requirements

The developer shall update the ModelProfile struct in Types.hpp to maintain two independent sets of Welford variables:

Fields for Preprocessing (J_{pre}):

- count_pre, mean_pre, m2_pre, stddev_pre

Fields for Online Inference (J_{on}):

- count_on, mean_on, m2_on, stddev_on

4. Scientific Significance for the Paper

This implementation allows us to claim **"Phase-Specific Stochastic Characterization"**:
"KairosSNNI distinguishes between the stochasticity of server-side resource preparation and the volatility of synchronous client-server interaction. By maintaining independent variance trackers for the offline and online phases, the Profiler provides a granular confidence interval for each stage of the pipeline. This enables the scheduler to identify whether an SLO risk is originating from local resource contention or remote client asymmetry."

Summary for Task List:

- **Refactor Types.hpp**: Split the single Welford tracker into two distinct instances within the ModelProfile struct.
- **Update Profiler.cpp**: Ensure the update() function identifies which phase is being recorded and applies the logic to the correct tracker.
- **Persistence**: Ensure both σ_{pre} and σ_{on} are included in the logs/dynamic_profile.cfg checkpointing logic (C-25).

This ensures the "Stochastic Scheduling" portion of your problem formulation is backed by rigorous, phase-aware empirical data.

FU-6: n_alt Batch Size Awareness Priority: Medium Context: config.cfg line 112, C-04 Question: The N_ALT_MAP in config.cfg maps batch-1 variants only (e.g., alexnet_1 -> simc2_1). If a client requests alexnet_8 (batch size 8), there is no mapping entry. Should alternatives preserve the original batch size (alexnet_8 -> simc2_8), always use batch 1, or is batch > 1 out of scope? Answer:

Regarding **FU-6: n_alt Batch Size Awareness**, follow these technical instructions to ensure the negotiation protocol maintains "Batch Integrity" while reducing resource pressure:

1. Decision: Preserve Original Batch Size

Option A is mandatory. The alternative model shift **must preserve the original batch size** submitted by the client (e.g., alexnet_8 →simc2_8).

2. Technical Reasoning

- **Batch Integrity:** SNNI clients prepare secret shares for a specific batch size (B_i) before transmission. If the server unilaterally changes the batch size to 1, the client's local reconstruction logic will fail, violating the "Black Box" protocol integrity.
- **Non-Linear Efficiency:** As seen in our profiling data (CSV), SNNI protocols are more efficient at higher batch sizes (e.g., alexnet_4 is not $4\times$ slower than alexnet_1). Dropping to batch 1 would actually decrease the system's total throughput and utility (Ψ).
- **The "Hostage Core" Goal:** The objective of n_{alt} is to reduce the **complexity per input**, not the number of inputs. Shifting from a "Wide" model (AlexNet) to a "Narrow" model (SimC2) at the *same* batch size still significantly reduces the server's core-occupancy time.

3. Implementation Requirements

The developer shall refactor the n_alt lookup logic to be **Batch-Relative**:

1. **Refactor config.cfg:** Instead of mapping specific instances (like alexnet_1), map the **Model Families**:
    - N_ALT_FAMILY_MAP: alexnet->simc2, vgg16->alexnet, simc2->hinet, hinet->simc1
2. **Dynamic Lookup:**
    - If a client requests alexnet_8 and $\theta_c \geq 1.5$:
    - The system identifies the family alternative is simc2.
    - It then constructs the lookup key: simc2 + _ + 8.
3. **Validation:**
    - The **Admission Control** must verify that the simc2_8 profile exists in profile.cfg.
    - If the alternative model does not support the requested batch size, the system should fall back to the **Hostage Core Penalty ($\phi = 1$)** for the original model.

4. Scientific Significance for the Paper

This ensures your "Integrated Partitioning" claim remains valid:

"KairosSNNI maintains **Batch-Level Consistency** during architectural negotiation. By shifting clients to narrower model families while preserving the requested batch size, the orchestrator minimizes the computational 'Privacy Tax' without requiring the client to re-encode their private data. This approach respects the non-linear scaling properties of FSS-based protocols while effectively mitigating core-occupancy bottlenecks."

Summary for Task List:

- **Update config.cfg**: Change the mapping to a family-based format (AlexNet → SimC2).
- **Update AdmissionControl.cpp**: Implement the batch-string concatenation for the n_alt proposal.
- **Error Handling**: If n_alt at batch B is not profiled, log a warning and proceed with the Penalty flag on the original request.

This preserves the "SNNI Specialty" (Batching logic) while making the negotiation protocol more robust and realistic for multi-tenant services.

FU-7: Lightweight Model Negotiation Behavior
Priority: Medium
Context: Q-3 negotiation state machine, config.cfg N_ALT_MAP
Question: The n_alt mapping only covers heavy models (AlexNet, VGG16, D4, ResNet50). If a client requests a lightweight model (e.g., simc1_1) and has theta_c >= 1.5, should negotiation trigger? If so, what are the alternatives? If not, should the request be processed without negotiation?
Answer:
Regarding **FU-7: Lightweight Model Negotiation Behavior**, follow these technical instructions to ensure the system handles "weak clients" on "small models" without wasting negotiation time.

1. Decision: Automatic Penalty for Lightweight "Bottom-of-Chain" Models

If a client is identified as weak ($\theta_c \geq 1.5$) but the requested model is already a **lightweight baseline** (e.g., simc1, m1, or hinet) with no defined n_{alt} in the mapping table:

- **Do NOT trigger the 200ms negotiation wait.**
- **Action:** Immediately apply the **Hostage Core Penalty ($\phi = 1$)**.
- **Reasoning:** Since there is no "lighter" model to shift to, negotiation is logically impossible. However, the client's slowness still creates a synchronous bottleneck on the server. Applying $\phi = 1$ ensures the **Online Scheduler** moves this slow task to the back of the queue to protect the SLOs of high-performance tenants.

2. Defining the "Lightweight" Boundary

The system shall treat any model that lacks an entry in the N_ALT_FAMILY_MAP (defined in FU-6) as a "Bottom-of-Chain" model.

Logic Flow for Admission Control:

1. **Check θ_c:** Is the client weak? ($\theta_c \geq 1.5$)
2. **Lookup n_{alt}:** Is there an alternative family for the requested model?
3. **Branching:**
    - **IF** n_{alt} exists: Initiate the 200ms Negotiation Protocol (propose shift).
    - **ELSE** (n_{alt} is null/missing): Skip negotiation. Set $\phi = 1$. Enqueue the request immediately.

3. Scientific Significance for the Paper

This logic prevents the "Negotiation Overhead" for tasks that cannot be optimized further: "KairosSNNI distinguishes between **Negotiable** and **Non-Negotiable Asymmetry**. For heavyweight models, the system actively attempts to reduce server-side core occupancy through architectural shifts.

For lightweight models where no further model reduction is feasible, the orchestrator applies an automatic **Hostage Core Penalty (ϕ)**. This identifies the request as a 'Slow-Lane' task, ensuring that even a lightweight job from a resource-constrained device does not induce Head-of-Line blocking for symmetric, latency-critical tenants."

Summary for Task List:

- **Update AdmissionControl.cpp**:
  - Add a check: if (theta_c >= 1.5 && n_alt_map.find(model_family) == n_alt_map.end()).
  - If true, set penalty_phi = 1 and proceed directly to enqueue().
- **Logging**: Ensure the Logger records Exit_Code or a new flag indicating the request was a "Direct Penalty" (Skipped Negotiation).
- **Config**: Ensure simc1 and other base models remain "unmapped" in config.cfg to trigger this logic.

This logic makes the **Admission Control** more efficient. It ensures we don't waste 200ms asking a client to "downgrade" from the smallest model in the catalog.

4.3 Low Priority (Future Planning)

FU-8: DQN Reward Function and Energy Priority: Low Context: train_dqn.py lines 82-86, contract page 13 Question: The current DQN reward function is purely SLO-based (+10 if met, -5 if not). The contract mentions extending to 'multi-objective task scheduling for energy efficient and cost effective.' Should the DQN reward incorporate Energy_uJ? If so, what weights for SLO vs. energy efficiency? Answer:

Regarding **FU-8: DQN Reward Function and Energy**, please follow these instructions for the Phase 1 implementation and future scaling.

1. Decision: Implement Multi-Objective Reward

Yes, the DQN reward must move beyond simple binary success. It should reflect the global optimization goal: maximizing the ratio of success to cost (Time and Energy).

2. Updated Reward Formula

Implement the following weighted reward function in train_dqn.py:

[Reward = (w_1 \cdot R_{\text{SLO}}) + (w_2 \cdot R_{\text{TAT}}) + (w_3 \cdot R_{\text{Energy}})]

Weights for Phase 1:

- $w_1 = 10.0$

(Primary): SLO attainment.

- $w_2 = 1.0$

(Secondary): Minimizing sojourn time (supports the denominator of Ψ).

- $w_3 = 0.1$**(Tertiary):** Energy efficiency.

3. Reward Logic Components

- R_{SLO}**:** $+1.0$ if $\chi = 1$ (met), -1.0 if $\chi = 0$(violated).
- R_{TAT}**:** $-(TAT/D_{i,k})$. This penalizes long waiting and execution times relative to the deadline.
- R_{Energy}**:** $-(Energy_{uJ}/\text{Baseline\_Energy})$. This penalizes high-power core configurations that don't result in significant SLO gains.

4. Scientific Significance for the Paper

This change allows us to claim **"Cost-Effective Private Inference"**:

"KairosSNNI utilizes a multi-objective reinforcement learning policy. By incorporating energy delta (Δ Joules) into the DQN reward function, the agent learns to select malleable core configurations that balance execution speed against the 'Privacy Tax' of high-density modular arithmetic. This ensures the system remains cost-effective in energy-constrained edge-cloud environments."

Summary for Task List:

- **Update train_dqn.py**: Refactor the reward loop to ingest Actual_TAT and Energy_uJ from the logs.
- **Configurability**: Move the weights w_1, w_2, w_3 to config.cfg under DQN_REWARD_WEIGHTS.
- **Logging**: Ensure the DQN_Reward value is logged in a separate training trace for analysis.

This aligns the DQN with the "multi-objective" requirement of the contract while ensuring that meeting deadlines remains the top priority.

Our students work with the following while doing performance evaluation of SNNI
Following links could be helpful: https://sourceforge.net/projects/iperf2/
For energy **Energy & Power Consumption Monitoring with scaphandre**
: https://medium.com/@salimbouaram12/energy-consumption-monitoring-with-scaphandre-5244f68f74c8

https://github.com/hubblo-org/scaphandre

FU-9: TTFR Baseline per Network Type Priority: Low Context: config.cfg line 103, Q-2 TTFR Handshake Question: TTFR_BASELINE_RTT_MS is set to 5ms (LAN). For clients on WiFi/5G/WAN, this will always produce theta_c >> 1.0, potentially triggering unnecessary negotiation. Should the baseline be differentiated by network type, or is the single 5ms value correct for the intended deployment scenario?
Answer:
Regarding **FU-9: TTFR Baseline Differentiation**, follow these technical instructions to ensure the **Telemetry Engine** accurately distinguishes between **hardware bottlenecks** and **network environments**.

1. Decision: Differentiated Network Baselines

The single 5ms (LAN) baseline is **rejected**. Using a static baseline for all connections would cause "False Positive" slowness detections for WAN/5G clients. The baseline must be differentiated based on the net_type field provided in the **Capability Vector (Cap_c)**.

2. Implementation Requirements

The developer shall update the Profiler and config.cfg to support a mapping of baseline RTT values.

Updated config.cfg schema:

```
# TTFR Baselines per Network Class (ms)
TTFR_BASE_LAN: 5.0
TTFR_BASE_WIFI: 20.0
TTFR_BASE_5G: 40.0
TTFR_BASE_WAN: 100.0
```

Revised θ_c

Calculation Logic:

When a request $R_{i,k}$ arrives, the system must:

1. Identify the net_type from the ingress string (0–3).
2. Retrieve the corresponding TTFR_BASE for that specific network.
3. Calculate the factor:
   [\theta_c = \max\left(1.0, \frac{\text{TTFR}_{\text{measured}}}{\text{TTFR}_{\text{base}}(\text{net_type})}\right)]

3. Scientific Reasoning for the Paper

This implementation allows us to isolate **Client Compute Slowness** from **Environment Latency**:

- **The Logic:** If a WAN client ($net = 3$) responds in 110ms, and the baseline for WAN is 100ms, $\theta_c \approx 1.1$. The system recognizes the client is computationally healthy for its environment.
- **The Goal:** Negotiation and the "Hostage Core Penalty" should only be triggered if the client is **slower than its own network class baseline**, indicating hardware-level resource constraints.

4. Impact on Resource Orchestration

Even though we differentiate the baseline, the **Online Scheduler** must still use the total expected duration for **Virtual Finish Time (VFT)**.

- **The Nuance:** θ_c now represents "Relative Hardware Efficiency."
- **Calculation:** The total execution time $E_{i,c}$ still accounts for the absolute network delay, but the "Penalty" ϕ and "Negotiation" are only triggered by hardware slowness relative to the network class.

Summary for Developer Task List:

- **Update config.cfg**: Replace the single 5ms constant with the 4-tier baseline map.
- **Modify Profiler.cpp**: Update the calculateTheta function to switch/case on the net_type from the request object.
- **Logging**: Ensure the Theta_Est column in the CSV is calculated using the network-corrected baseline.

This makes the "Asymmetry-Aware" logic scientifically sound. It prevents KairosSNNI from unfairly penalizing WAN users who have strong hardware but high-latency connections.

FU-10: Phase Roadmap Beyond Phase 1 Priority: Low Context: C-01, Q-13 Question: The contract references 'Phase 5' (C-01, page 17) but does not define Phases 2-4. Understanding the roadmap would help ensure Phase 1 implementation does not create architectural debt for later phases. Is a formal phase breakdown available? Answer:

Regarding **FU-10: Phase Roadmap**, the project follows a logical progression from "Functional Core" to "System Intelligence." While Phase 1 delivers the full functional framework, the subsequent phases focus on refining the adaptive logic and measuring the "Privacy-Efficiency Trade-off" for the research paper. The following roadmap defines the milestones to prevent architectural debt:

Phase 1: Functional SISE Orchestrator (Current)

- **Goal:** Build the "Muscle" and the "Plumbing."

- **Deliverables:** 8-field TCP gateway, 21-column CSV logging, and the basic Request Scheduler (FCFS/EDF).
- **Key Logic:** Integrated "Triple Check" and non-preemptive job launching.

Phase 2: Workload & Stress Resilience

- **Goal:** Test the "Memory Wall" under pressure.
- **Focus:** Implementing the **Poisson Benchmarking Harness** ($\lambda = 0.1 \ldots 2.0$).
- **Logic:** Proving **HoL (Head-of-Line) Mitigation** and 1TB Project Space stability during high-load bursts of VGG16 models.

Phase 3: Adaptive Asymmetry Management

- **Goal:** Implement the "Closed-Loop" feedback.
- **Focus: Negotiation State Machine** and **EWMA-based Profiler updates**.
- **Logic:** This phase enables the system to handle "Weak Clients" by suggesting model shifts (n_{alt}) or applying the **Hostage Core Penalty (ϕ)**.

Phase 4: Efficiency & Energy Profiling

- **Goal:** Measure the "Privacy Tax" in Joules.
- **Focus: Scaphandre integration** and multi-objective **DQN Reward** refinement (w_3 energy weight).
- **Logic:** Shifting from "Time-only" scheduling to "Success-per-Watt" optimization.

Phase 5: The Scheduling Tournament & Final Audit

- **Goal:** Generate the final scientific results.
- **Focus:** Running 1,000-episode tests for **Simulated Annealing** and **DQN**.
- **Deliverable:** Comparative analysis of Ψ **(Psi)** across all 6+1 algorithms to identify the "Crossover Points" for publication.

Summary for Architectural Planning:

- Ensure the Profiler and OnlineScheduler classes are modular. We will add more "weights" to their cost functions in Phase 4.
- Ensure the Logger can handle the expanded 21-column trace immediately (as per C-10) to avoid refactoring the database in Phase 5.
- The **"Mask-Burn" policy** is a Phase 1 requirement to ensure Phase 3 (Negotiation) is secure from day one.

This roadmap ensures that Phase 1 is a solid foundation for the complex AI and energy modules coming in the next six weeks.

FU-10: Phase Roadmap Beyond Phase 1
Priority: Low
Context: C-01, Q-13
Question: The contract references 'Phase 5' (C-01, page 17) but does not define Phases 2-4. Understanding the roadmap would help ensure Phase 1 implementation does not create architectural debt for later phases. Is a formal phase breakdown available?
Answer:

Regarding **FU-10: Phase Roadmap Beyond Phase 1**, please note that the project timeline has been consolidated. **Phase 1 represents the single, full functional delivery of the complete KairosSNNI orchestration framework.**

There are no subsequent development phases. To satisfy the research requirements and the fixed-price agreement, you are required to deliver the entire integrated system in this initial delivery. All architectural components previously discussed as "future phases" must be fully implemented now to prevent architectural debt and ensure the system is research-ready.

Phase 1: Full Technical Requirements for Delivery

You must deliver the **Secure Inference Service Engine (SISE)** with the following functional modules fully integrated:

1. Comprehensive Algorithm Suite (Request Scheduler):

- Implementation of all **6+1 Scheduling Modes**: Baselines (**FCFS, SJF-Aging**), Urgency-Based (**EDF, LST**), and Intelligent Optimizers (**Simulated Annealing** with 3D perturbation and **DQN** with Pointwise Ranking policy).
- Integration of **Malleable Core Sizing** ($1 \dots 16$ cores) and **Amdahl-based non-linear speedup modeling** ($k = 0.8$) within the SA and DQN decision logic.

2. Asymmetry-Aware Admission Control:

- **The Three-Tier SLO Filter:** Strict sequential execution of the Practicality Filter (Tier 1), Static SLO-Factor Filter (Tier 2), and the Dynamic **Virtual Finish Time (VFT)** Filter (Tier 3).
- **The Negotiation State Machine:** Full implementation of model-shift proposals (n_{alt}), 200ms asynchronous client timeouts, and the **Priority Penalty ($\phi = 1$)** for negotiation refusals.
- **Security Protocol:** Enforcement of the **Mask-Burn policy** (material invalidation) during model shifts to maintain FSS protocol integrity.

3. Advanced Telemetry and Learning Engine:

- **Two-Stage Verification: Timer 1** (TCP-based TTFR Handshake Probe) for initial θ_c estimation and **Timer 2** (External ReLU Execution Wrapper) for ground-truth verification.
- **Stochastic Profiling:** Implementation of **EWMA** smoothing and **Welford's Algorithm** to track mean and standard deviation (σ) for both protocol phases (J_{pre} and J_{on}).
- **Energy Metrology:** Integration of **Scaphandre** for process-level energy tracking ($Energy_{uJ}$) in the 21-column CSV trace.

4. Resource Orchestration and Stability:

- **Memory Management:** Continuous **100ms background memory polling** and a strict **90% RAM Dispatcher Cap** with a 2GB hard reserve to manage the 35GB VGG-16 "Memory Wall."
- **I/O Management:** Full coordination with the **1TB Project Space** for J_{pre} material staging and JIT (Just-in-Time) promotion to RAM.
- **Performance Isolation: Strict Physical Core Pinning** and **Non-blocking HoL Mitigation** (bypassing requests with missing dependencies).

Final Acceptance Criteria (The Scheduling Tournament)

Delivery is considered complete only when the system can execute the **run_kairos_tournament.sh** script. For instance, this script should:

1. Generate a **1,000-request Poisson burst** ($\lambda = 1.0$) with a 70/20/10 model mix.

2. Compare all scheduling modes and output the **Global Utility Metric (Ψ)** for each.
3. Prove that **KairosSNNI (SA/DQN)** achieves the 20% improvement target in Ψ over the FCFS/SJF+agind/EDF/LST baselines.
4. Produce a valid, 21-column CSV trace and a 100ms time-series memory log.

Summary:

The goal of this delivery is a **Complete Research Artifact**. Please ensure all 32 technical decisions (C-01 to C-32) are reflected in the source code. All placeholders must be replaced with operational logic as defined in this consolidated Phase 1 scope.

The future work will be related to another similar SNNI work.